```
# Cell 1 — imports
import nltk; nltk.download('punkt', quiet=True)
import os, time, math, random
import numpy as np, matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunctio
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

```
# Cell 2 — upload dataset
if not os.path.exists("vast_english_french.txt"):
    from google.colab import files
    print("Please choose vast_english_french.txt ..."); files.upload()
print("Dataset present:", os.path.exists("vast_english_french.txt"))
```

```
Please choose vast_english_french.txt ...
Choose Files vast_english_french.txt
vast_english_french.txt(text/plain) - 42659 bytes, last modified: n/a - 100% done
Saving vast_english_french.txt to vast_english_french.txt
Dataset present: True
```

```
# Cell 3 — vocab/split/tensors (REVERSED: French=source, English=targe
PAD_token, SOS_token, EOS_token, UNK_token = 0, 1, 2, 3
class Vocabulary:
    def __init__(self):
        self.word2index = {"<PAD>":0,"<SOS>":1,"<EOS>":2,"<UNK>":3}
        self.index2word = {0:"<PAD>",1:"<SOS>",2:"<EOS>",3:"<UNK>"}; s
    def add_sentence(self, s):
        for w in s.split(' '): self.add_word(w)
    def add_word(self, w):
        if w not in self.word2index:
            self.word2index[w]=self.n_words; self.index2word[self.n_wo
    def encode(self, s, max_len):
        ids=[SOS_token]+[self.word2index.get(w,UNK_token) for w in s.s
        ids=ids[:max_len]; return ids+[PAD_token]*(max_len-len(ids))
def load_pairs(path):
    pairs=[]
    with open(path, encoding="utf-8") as f:
        for line in f:
            line=line.rstrip("\n")
            if not line.strip(): continue
            p=line.split("\t")
            if len(p)==2 and p[0].strip() and p[1].strip(): pairs.appe
    return pairs
all_pairs=load_pairs("vast_english_french.txt")  # (english, french)
rng=random.Random(SEED); idx=list(range(len(all_pairs))); rng.shuffle
split=int(0.8*len(all_pairs))
train_pairs=[all_pairs[i] for i in idx[:split]]; val_pairs=[all_pairs
print(f"Train: {len(train_pairs)} | Val: {len(val_pairs)}")
fr_vocab, eng_vocab = Vocabulary(), Vocabulary()
for eng,fr in train_pairs: fr_vocab.add_sentence(fr); eng_vocab.add_se
print(f"FR vocab: {fr_vocab.n_words} | EN vocab: {eng_vocab.n_words}"
MAX_LEN=max(max(len(e.split()) for e,_ in all_pairs), max(len(f.split
def make_tensors(pairs, ml):
    src=torch.tensor([fr_vocab.encode(f,ml)  for _,f in pairs],dtype=t
    tgt=torch.tensor([eng_vocab.encode(e,ml) for e,_ in pairs],dtype=t
train_src,train_tgt=make_tensors(train_pairs,MAX_LEN)
val_src,val_tgt=make_tensors(val_pairs,MAX_LEN)
```

```
Train: 444 | Val: 111
FR vocab: 1004 | EN vocab: 903
```

```
# Cell 4 — both architectures
class EncoderGRU(nn.Module):
    def __init__(self, input_size, hidden_size, dropout=0.1):
        super().__init__(); self.hidden_size=hidden_size
        self.embedding=nn.Embedding(input_size,hidden_size,padding_id
        self.dropout=nn.Dropout(dropout); self.gru=nn.GRU(hidden_size,
    def forward(self, x, h):
        emb=self.dropout(self.embedding(x)).unsqueeze(1); out,h=self.g
    def initHidden(self): return torch.zeros(1,1,self.hidden_size,dev
class DecoderGRU(nn.Module):  # baseline
    def __init__(self, hidden_size, output_size, dropout=0.1):
        super().__init__()
        self.embedding=nn.Embedding(output_size,hidden_size,padding_id
        self.dropout=nn.Dropout(dropout); self.gru=nn.GRU(hidden_size,
        self.out=nn.Linear(hidden_size,output_size); self.log_softmax=
    def forward(self, x, h, enc_outputs=None):
        emb=self.dropout(self.embedding(x)).view(1,1,-1); o,h=self.gru
        return self.log_softmax(self.out(o[0])), h, None
class BahdanauAttention(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.Wa=nn.Linear(hidden_size,hidden_size,bias=False)
        self.Ua=nn.Linear(hidden_size,hidden_size,bias=False)
        self.Va=nn.Linear(hidden_size,1,bias=False)
    def forward(self, dec_hidden, enc_outputs):
        q=self.Wa(dec_hidden.squeeze(0)); k=self.Ua(enc_outputs.squeez
        scores=self.Va(torch.tanh(q+k)); w=torch.softmax(scores,dim=0)
        ctx=(w*enc_outputs.squeeze(1)).sum(0,keepdim=True); return ctx
class AttnDecoderGRU(nn.Module):
    def __init__(self, hidden_size, output_size, dropout=0.1):
        super().__init__()
        self.embedding=nn.Embedding(output_size,hidden_size,padding_id
        self.dropout=nn.Dropout(dropout); self.attention=BahdanauAtten
        self.gru=nn.GRU(hidden_size*2,hidden_size); self.out=nn.Linear
        self.log_softmax=nn.LogSoftmax(dim=1)
    def forward(self, x, h, enc_outputs):
        emb=self.dropout(self.embedding(x)).view(1,1,-1)
        ctx,w=self.attention(h,enc_outputs)
        gin=torch.cat([emb,ctx.view(1,1,-1)],dim=2); o,h=self.gru(gin,
        return self.log_softmax(self.out(o[0])), h, w
```

```
# Cell 5 — helpers + train_model (best-BLEU)
def seq_loss(enc, dec, src, tgt, crit):
```

```
        h=enc.initHidden(); sl=(src!=PAD_token).sum().item(); eo_,h=enc(s
        di=torch.tensor([[SOS_token]],device=device); dh=h; loss,n=0.0,0
        tl=(tgt!=PAD_token).sum().item()
        for t in tgt[1:tl]:
            o,dh,_=dec(di,dh,eo_); loss=loss+crit(o,t.unsqueeze(0)); n+=1
        return loss, max(n,1)
    def run_epoch(enc, dec, st, tt, crit, eo=None, do=None, train=True):
        enc.train(train); dec.train(train); total,count=0.0,0
        order=list(range(st.size(0)))
        if train: random.shuffle(order)
        ctx=torch.enable_grad() if train else torch.no_grad()
        for i in order:
            src,tgt=st[i].to(device),tt[i].to(device)
            if train: eo.zero_grad(); do.zero_grad()
            with ctx: loss,n=seq_loss(enc,dec,src,tgt,crit)
            if train:
                loss.backward(); nn.utils.clip_grad_norm_(enc.parameters(
                nn.utils.clip_grad_norm_(dec.parameters(),5.0); eo.step()
            total+=loss.item(); count+=n
        return total/count
    def translate(enc, dec, src, max_len=MAX_LEN):
        enc.eval(); dec.eval()
        with torch.no_grad():
            h=enc.initHidden(); sl=(src!=PAD_token).sum().item(); eo_,h=en
            di=torch.tensor([[SOS_token]],device=device); dh=h; out=[]
            for _ in range(max_len):
                o,dh,_=dec(di,dh,eo_); _,topi=o.topk(1); ix=topi.item()
                if ix==EOS_token: break
                if ix!=PAD_token: out.append(ix)
                di=topi.detach().view(1,1)
        return out
    def ids_to_words(ids,v): return [v.index2word.get(i,"<UNK>") for i in
    def evaluate(enc,dec,pairs,st):
        sm=SmoothingFunction().method1; ex,tb=0,0.0
        for i,(eng,fr) in enumerate(pairs):
            pred=ids_to_words(translate(enc,dec,st[i]),eng_vocab); ref=eng
            tb+=sentence_bleu([ref],pred,smoothing_function=sm) if pred e
        n=len(pairs); return ex/n,tb/n
    def train_model(decoder_cls, tag):
        HIDDEN,LR,EPOCHS,PATIENCE=256,0.001,60,12
        enc=EncoderGRU(fr_vocab.n_words,HIDDEN,dropout=0.1).to(device)
        dec=decoder_cls(HIDDEN,eng_vocab.n_words,dropout=0.1).to(device)
        crit=nn.NLLLoss(); eo=optim.Adam(enc.parameters(),lr=LR); do=optim
        sm=SmoothingFunction().method1
        def vbleu():
```

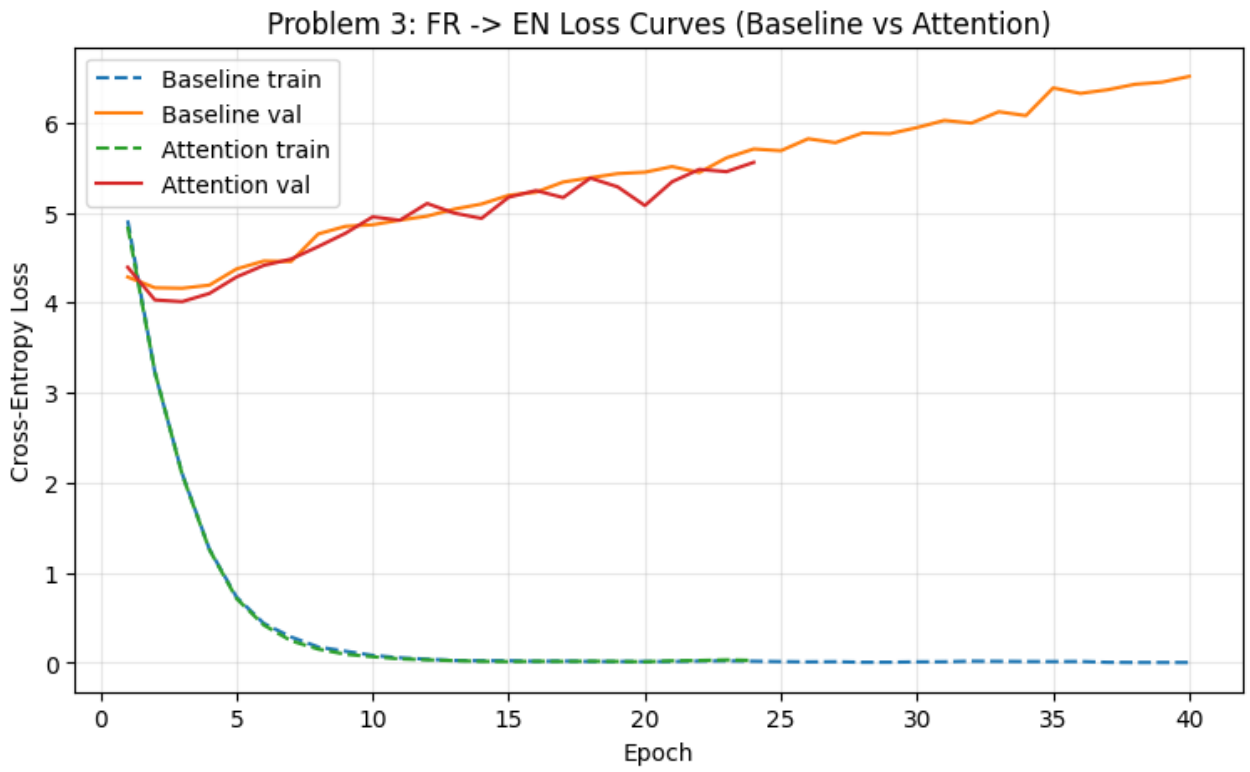
```
        tot=0.0
        for i,(eng,fr) in enumerate(val_pairs):
            pred=ids_to_words(translate(enc,dec,val_src[i]),eng_vocab
            ref=eng.split(' '); tot+=sentence_bleu([ref],pred,smoothi
        return tot/len(val_pairs)
    tr_h,va_h=[],[]; best_bleu,best_state,bad=-1.0,None,0
    for epoch in range(EPOCHS):
        tr=run_epoch(enc,dec,train_src,train_tgt,crit,eo,do,True)
        va=run_epoch(enc,dec,val_src,val_tgt,crit,train=False)
        vb=vbleu(); tr_h.append(tr); va_h.append(va)
        if vb>best_bleu:
            best_bleu,bad=vb,0
            best_state=({k:v.cpu().clone() for k,v in enc.state_dict(
                        {k:v.cpu().clone() for k,v in dec.state_dict(
        else: bad+=1
        if (epoch+1)%5==0 or epoch==0: print(f"[{tag}] Epoch {epoch+1
        if bad>=PATIENCE: print(f"[{tag}] Early stop at epoch {epoch+1
    enc.load_state_dict(best_state[0]); dec.load_state_dict(best_state
    print(f"[{tag}] best val BLEU: {best_bleu:.4f}")
    return enc,dec,tr_h,va_h
```

```
# Cell 6 — train BOTH models
print("===== Baseline GRU (FR -> EN) =====")
b_enc,b_dec,b_tr,b_va=train_model(DecoderGRU,"baseline")
print("\n===== Attention GRU (FR -> EN) =====")
a_enc,a_dec,a_tr,a_va=train_model(AttnDecoderGRU,"attention")
```

```
===== Baseline GRU (FR -> EN) =====
[baseline] Epoch 01 train=4.9125 val=4.2848 valBLEU=0.0636 (bestBLEU=0
[baseline] Epoch 05 train=0.7288 val=4.3741 valBLEU=0.0810 (bestBLEU=0
[baseline] Epoch 10 train=0.0827 val=4.8650 valBLEU=0.1149 (bestBLEU=0
[baseline] Epoch 15 train=0.0232 val=5.1923 valBLEU=0.1234 (bestBLEU=0
[baseline] Epoch 20 train=0.0137 val=5.4484 valBLEU=0.1272 (bestBLEU=0
[baseline] Epoch 25 train=0.0134 val=5.6875 valBLEU=0.1214 (bestBLEU=0
[baseline] Epoch 30 train=0.0106 val=5.9442 valBLEU=0.1150 (bestBLEU=0
[baseline] Epoch 35 train=0.0134 val=6.3848 valBLEU=0.1122 (bestBLEU=0
[baseline] Epoch 40 train=0.0038 val=6.5124 valBLEU=0.1122 (bestBLEU=0
[baseline] Early stop at epoch 40
[baseline] best val BLEU: 0.1285

===== Attention GRU (FR -> EN) =====
[attention] Epoch 01 train=4.8465 val=4.3935 valBLEU=0.0527 (bestBLEU=(
[attention] Epoch 05 train=0.7131 val=4.2854 valBLEU=0.1079 (bestBLEU=(
[attention] Epoch 10 train=0.0673 val=4.9527 valBLEU=0.1151 (bestBLEU=(
[attention] Epoch 15 train=0.0131 val=5.1711 valBLEU=0.1338 (bestBLEU=(
[attention] Epoch 20 train=0.0115 val=5.0781 valBLEU=0.1296 (bestBLEU=(
[attention] Early stop at epoch 24
[attention] best val BLEU: 0.1381
```

```
# Cell 7 — overlaid loss curves
plt.figure(figsize=(9,5))
plt.plot(range(1,len(b_tr)+1),b_tr,"--",label="Baseline train")
plt.plot(range(1,len(b_va)+1),b_va,"-",label="Baseline val")
plt.plot(range(1,len(a_tr)+1),a_tr,"--",label="Attention train")
plt.plot(range(1,len(a_va)+1),a_va,"-",label="Attention val")
plt.title("Problem 3: FR -> EN Loss Curves (Baseline vs Attention)")
plt.xlabel("Epoch"); plt.ylabel("Cross-Entropy Loss"); plt.legend();
```



```
# Cell 8 — metrics table + samples
b_acc,b_bleu=evaluate(b_enc,b_dec,val_pairs,val_src)
a_acc,a_bleu=evaluate(a_enc,a_dec,val_pairs,val_src)
print("=== Problem 3 Validation Metrics (FR -> EN) ===")
print(f"{'Model':<12}{'Seq Acc %':>12}{'BLEU-4':>10}")
print(f"{'Baseline':<12}{b_acc*100:>12.2f}{b_bleu:>10.4f}")
print(f"{'Attention':<12}{a_acc*100:>12.2f}{a_bleu:>10.4f}\n")
print("=== Qualitative FR -> EN Samples ===")
for j in range(5):
    eng,fr=val_pairs[j]
    bp=' '.join(ids_to_words(translate(b_enc,b_dec,val_src[j]),eng_voca
    ap=' '.join(ids_to_words(translate(a_enc,a_dec,val_src[j]),eng_voca
    print(f"\nFR  : {fr}\nREF : {eng}\nBASE: {bp}\nATTN: {ap}")
```

```
=== Problem 3 Validation Metrics (FR -> EN) ===
Model          Seq Acc %    BLEU-4
Baseline            0.00    0.1285
Attention           0.00    0.1381

=== Qualitative FR -> EN Samples ===

FR  : Ils nourrissent les pigeons sur la place
REF : They feed the pigeons in the square
BASE: They play volleyball on the beach
ATTN: They go the instructions carefully to avoid mistakes

FR  : Elle pratique le yoga tous les matins
REF : She practices yoga every morning
BASE: She practices the violin every day
ATTN: She practices the cello for three hours every day

FR  : J'aime marcher dans la neige
REF : I enjoy walking in the snow
BASE: I enjoy walking in the rain
ATTN: I enjoy walking in the rain

FR  : Elle adore porter des vestes modernes
REF : She loves to wear modern jackets
BASE: She loves to wear modern leather jackets
ATTN: She loves to wear modern leather jackets

FR  : Le bus de la ville arrive précisément à cinq heures
REF : The city bus arrives precisely at five o'clock
BASE: The bus arrives at five o'clock
ATTN: The bus arrives at five o'clock
```